

Local RAG System

NLP Project — Technical Report

FastAPI · ChromaDB · Ollama (Gemma 3) · Docker

2023170599 — مصطفى احمد نبيل ماهر احمد عيد
2023170703 — ياسين هاني ماجد محمد مروان
2023170702 — ياسين مصطفى وهب محمد راشد
2023170751 — يوسف وائل محمد محمود
2022170125 — حبيبة أسامة سيد محمد

1. Executive Summary

A fully containerized RAG system that ingests unstructured documents (PDFs, DOCX), embeds them with sentence-transformers, stores vectors in ChromaDB, and answers natural-language queries by injecting retrieved context into a local LLM (Gemma 3 via Ollama) — all served through a FastAPI backend and deployed with a single docker-compose up command. The use case is CV querying: given a messy, unstructured CV, the system answers recruiter-style questions about the candidate.

2. System Architecture

The pipeline follows a strict MVC separation across dedicated modules. On ingestion, parser.py extracts text page-by-page from PDFs (PyMuPDF) or paragraph-by-paragraph from DOCX files (python-docx), attaching source and page metadata to each chunk. chunker.py then splits the text using LangChain's RecursiveCharacterTextSplitter and embedder.py generates 768-dimensional vectors using BAAI/bge-base-en-v1.5, which are upserted into a ChromaDB persistent collection with cosine similarity.

At query time, routers/query.py receives the request and calls rag_service.py, which embeds the query, retrieves the top-k most similar chunks from ChromaDB via retriever.py, injects them into a prompt, and sends the prompt to Ollama's HTTP API (gemma3 model). The structured response — answer + source chunks + scores — is returned via a Pydantic model.

3. Chunking & Embedding Justification

Documents are chunked with a size of 500 characters and 50-character overlap. The 500-char limit is large enough to hold a complete semantic unit (e.g. a full work experience entry) while staying within embedding model limits. The 50-char overlap prevents boundary losses — a job title and its company name will always co-appear in at least one chunk. The splitter prioritizes paragraph breaks, then sentences, then words, so it respects semantic units rather than applying hard cuts.

The embedding model was upgraded from all-MiniLM-L6-v2 (384-dim) to BAAI/bge-base-en-v1.5 (768-dim) after Phase 4 evaluation revealed the smaller model failed to bridge the vocabulary gap between natural-language queries and CV text. BGE-base is specifically optimized for retrieval tasks and runs on CPU without a GPU.

4. API Documentation

POST /api/query

Accepts a natural-language query and returns an LLM-generated answer with retrieved source chunks and similarity scores.

Request	Response
<pre>{ "query": "Where does the candidate work?", "top_k": 3}</pre>	<pre>{ "query": "...", "answer": "Instabug, Cairo", "chunks": [{ "text": "...", "source": "cv.pdf", "page": 1, "score": 0.51 }]}</pre>

GET /api/health

Returns { "status": "ok" } — used to verify the container is running.

5. Docker Deployment

```
git clone <repo-url> && cd rag-project
docker-compose up --build
# API at http://localhost:8000/docs
```

Two services start: rag-api (FastAPI + ChromaDB + embedder) on port 8000, and ollama (Gemma 3 inference) on port 11434. The chroma_db/ volume is pre-ingested and mounted — no manual ingestion step is needed. First run downloads Gemma 3 (~3.3 GB); subsequent runs are instant.

6. Evaluation & Error Analysis

The system was tested with targeted edge-case queries against the ingested CV. Three distinct failure modes were identified:

- **Numerical aggregation failure: "How many years of experience does the candidate have?"**
- The retriever returned raw date ranges from individual job entries (2013–2018 at Raya IT, 2022–Present at Paymob) rather than a pre-computed summary of total experience. As a result, the LLM reproduced the literal date strings instead of calculating a coherent total, producing an answer like "2013–2018 years of experience @ Raya IT and 2022–Present years of experience @ Paymob" rather than a numeric figure (e.g., "~9 years total across two roles, with a gap from 2018–2022 for education and other activities").
- **Root cause:** the query "how many years" requires arithmetic aggregation across multiple chunks, but the retriever treats each chunk independently and returns no span that explicitly states a total. The semantic overlap between a years-counting query and fragmented work-history bullet points is too weak to surface a pre-aggregated answer, and none exists in the CV anyway.
- **Fix:** apply hybrid BM25 + vector retrieval to improve recall across all experience-related chunks, then add a post-retrieval aggregation step that extracts structured date spans, computes durations, and sums them before passing context to the LLM. Alternatively, enrich the index at ingestion time with a synthetic summary field that pre-computes total experience per candidate.

Ordinal reasoning failure: "What projects did the candidate work on at their second job?"

- The retriever returned projects from Raya IT (2013–2018), which is actually the candidate's first job, along with an unrelated freelance chunk and the CV header. The correct answer — Paymob (2022–Present), the second formal employer — was not retrieved at all, because embedding similarity has no concept of chronological order. The query term "second job" carries no semantic signal that maps to any specific employer name or date range in the chunks; the retriever simply surfaced the highest-scoring work-experience chunk regardless of its position in the candidate's timeline.
- **Root cause:** RAG ranks chunks by embedding similarity, not by the sequential position of a job entry within a document. Ordinal qualifiers like "second," "most recent," or "previous" are lost entirely during retrieval — the model has no structured representation of job order to reason against.
- **Fix:** at ingestion time, parse work experience entries and tag each chunk with structured metadata — specifically a `job_order` field (e.g., 1, 2, 3 in chronological order) and an `entry_type` field (e.g., `work_experience`). At query time, detect ordinal intent and translate it into a metadata filter (e.g., `job_order == 2`) applied before or alongside vector search, so the retriever is constrained to the correct entry before any similarity ranking occurs.

Semantic negation failure: "What programming languages does the candidate NOT know?"

The correct skills chunks were retrieved, but the LLM failed to apply the negation constraint correctly. The answer included Bash and JavaScript — both explicitly listed as known skills in the retrieved chunks — while also including C (marked "very rusty"), which is a borderline case but still a listed skill. The query was asking for languages absent from the CV entirely, yet the model conflated weak proficiency with not knowing a language, and hallucinated no distinction between "listed but weak" and "not listed at all."

Root cause: embedding models encode semantic similarity, not logical negation. The query "does NOT know" maps to the same embedding neighborhood as "knows," so retrieval behaves identically for both. The negation constraint is then passed to the LLM as plain context, and without explicit reasoning scaffolding, the model tends to pattern-match the most salient items in the chunk — the listed languages — rather than reason about their absence or apply the NOT filter rigorously.

Fix: apply chain-of-thought prompting to force the model to reason about negation in explicit steps before producing an answer. For example, instruct the model to first extract all languages the candidate does know from the retrieved chunks, then identify which languages were queried or implied, and only then produce a final answer about what is absent. A structured prompt step like "List all languages mentioned → now identify what is NOT in that list" prevents the model from short-circuiting directly to surface-level pattern matching. Optionally, complement this with a post-generation verification step that cross-checks the answer against the extracted skills list to catch any listed item that was incorrectly included.